

De recuperar texto a construir un sistema auditable

MIRIAM DEL BLANCO · ANDREA LABÁ

Routing por herramientas · retrieval híbrido · guardrail XBRL · evaluación reproducible

La respuesta no basta: medimos el camino, la cita, el coste y la latencia.

Pregunta



Routing



Fuente



Respuesta



Verificación

¿Qué recibe realmente el agente?

Los datos ya condicionan el diseño: el texto está fragmentado y XBRL es preciso, pero incompleto.

01 TEXTO + RETRIEVAL

48 secciones · 1.749 chunks

Cada chunk conserva

ticker · fiscal_year · item · chunk_id
posición · nº tokens · contiene_tabla

Tablas partidas entre chunks

El corte puede dejar fuera filas, cabeceras o la escala “(\$ in millions)”. Por eso un chunk correcto puede seguir siendo insuficiente.

Si falta contexto para entender la tabla se recupera la sección completa.

FAISS · 1.749 vectores · 384 dimensiones

02 XBRL

135 hechos · cobertura desigual

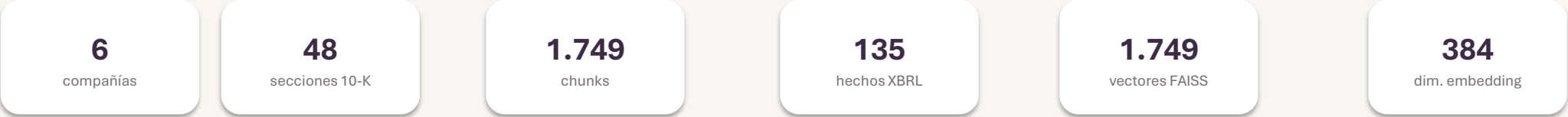
La mayoría de conceptos están en ambos ejercicios, pero hay excepciones relevantes:

GrossProfit	no AMZN · GOOGL · META
Liabilities	no AMZN
R&D Expense	no AMZN
RevenueFromContract...	GOOGL: 1 año · NVDA: 0
Revenues	solo GOOGL · NVDA

Un tag ausente no implica dato ausente: una misma magnitud puede cambiar de concepto según compañía.

Diseño resultante: XBRL para precisión · retrieval para significado · read_section cuando el chunk pierde contexto.

Routing asimétrico: cada tool tiene una función distinta



Los datos están alineados por `chunk\_id`: texto ↔ metadatos ↔ vector FAISS. Una desalineación sería un error silencioso.

get_xbri_fact

CIFRAS EXACTAS

Ejemplo

NVDA · FY2024 · Revenues

60.922.000.000 USD

XBRL

search_filings

RIESGOS / CAUSAS / ESTRATEGIA

Ejemplo

“operating income by reportable segments”

→ 5 chunks + chunk_id

retrieval

read_section

CONTEXTO COMPLETO

Ejemplo

si el chunk corta una tabla y era necesaria entera

→ sección completa

fallback caro

list_available

COBERTURA

Ejemplo

¿hay TSLA FY2025?

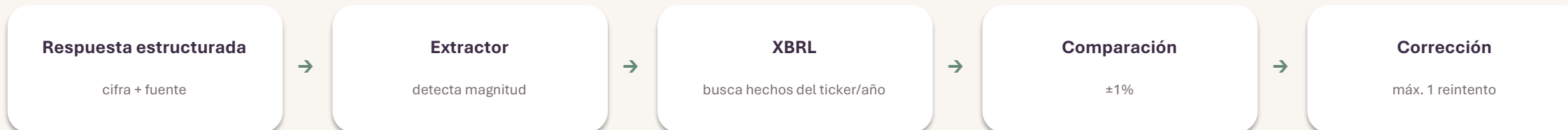
→ compañías/años/items

antes de negar

Idea de routing: cifra → XBRL · contenido cualitativo → retrieval · contexto insuficiente → read_section · cobertura dudosa → list_available

El control que pretendía evitar alucinaciones llegó a crear una

Flujo de validación



Caso que rompió el sistema

Meta · provisión por impuestos

1. El concepto no existía en XBRL.
2. ``search_filings`` encontró la cifra correcta en texto.
3. El middleware la comparó contra XBRL igualmente.
4. La rechazó y el agente se retractó.

Arreglo

Verificar según procedencia

Si ``fuente == XBRL`` → contraste automático.
Si la cifra viene de una cita textual → no exigir que exista como hecho XBRL.

Principio: el guardrail debe validar la fuente declarada, no imponer una fuente universal.

Qué mueve realmente el retrieval

Qué ocurre en `search\_filings`

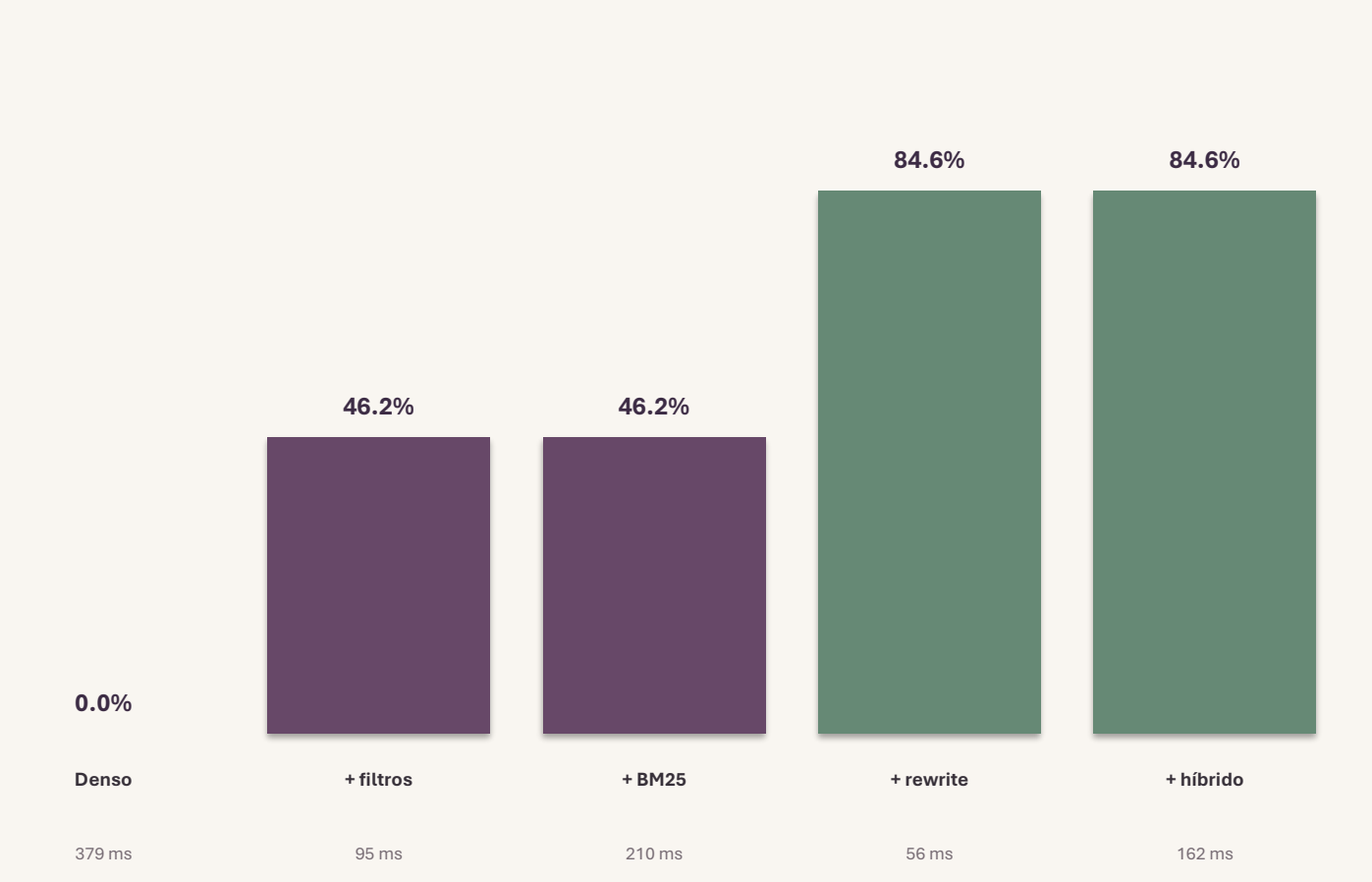
- 1 Filtros ticker · FY · item
- 2 Rewrite pregunta → consulta breve en inglés
- 3 Denso embedding 384D → FAISS
- 4 BM25 coincidencia léxica
- 5 RRF (híbrido) fusiona rankings, no scores

Ejemplo de chunk recuperado

[NVDA-2024-7-0014]
Compute & Networking \$47,405M
Graphics \$13,517M
Total \$60,922M

El `chunk\_id` permite volver al texto exacto y verificar la cita.

Ablación · Recall@5



84,6% = ablación de una búsqueda. El agente end-to-end puede iterar y alcanza 92,3% en el golden final.

Baseline vs. final · mismo golden set, mismo modelo

Mismo golden set y mismo modelo. El baseline es el agente de la sesión 1; el final incorpora routing afinado, retrieval mejorado y guardrails.

Métrica	Baseline	Final	Lectura
Cita	75,0%	100,0%	+25 pp
Recall@5	46,2%	92,3%	+46,1 pp
Cifra	100,0%	100,0%	se mantiene
Trayectoria	100,0%	100,0%	se mantiene
Coste / pregunta	\$0,01295	\$0,01273	-1,7%
Latencia / pregunta	8,16 s	10,99 s	+34,6%
Llamadas / pregunta	4,55	4,15	-8,8%

La mejora principal está en recuperación y citas. El coste medio incluso baja ligeramente; la contrapartida de esta ejecución es más latencia.

Lo que no funcionó — y por qué importa

1 · BM25 sin rewrite

46,2% → 46,2%

Añadir coincidencia léxica no solucionó el problema de recuperación: la consulta seguía utilizando un vocabulario distinto al del filing.

Qué aprendimos

Primero hay que alinear la consulta con el corpus. En esta ejecución, el salto llegó con el query rewrite.

BM25/RRF puede complementar rankings, pero aquí no añadió Recall@5.

2 · Guardrail XBRL

dato correcto → rechazado

En Meta, la provisión por impuestos estaba correctamente recuperada del filing, pero no existía bajo el concepto XBRL esperado.

Qué fallaba

El middleware obligaba a contrastar cualquier cifra contra XBRL. Si XBRL no la contenía, terminaba invalidando una respuesta correcta.

Arreglo: validar según la fuente declarada.

3 · Tablas y evaluación

el contexto importa

Una tabla puede conservar el valor “60,922” pero perder la cabecera “(\$ in millions)”. El número es correcto; la magnitud, no.

Además...

- `cifra` es ambiguo en preguntas derivadas.
- El split 10:1 de NVIDIA exige decidir qué convención espera el golden.

El evaluador también necesita una especificación precisa.

Los fallos más útiles no fueron del LLM: revelaron supuestos incorrectos en retrieval, validación y evaluación.

Resultados sobre el conjunto oficial

Aquí incorporaremos los resultados oficiales después de ejecutar las preguntas ciegas, sin modificar el sistema.

CITA

—

CIFRA

—

TRAYECTORIA

—

RECALL@5

—

RESULTADOS / OBSERVACIONES

Objetivo: comprobar si las mejoras observadas en el golden propio generalizan a preguntas no vistas.

Qué hemos aprendido y qué haríamos después

Qué nos llevamos

- Retrieval no es la arquitectura: es una herramienta dentro del agente.
- El mayor cuello de botella era localizar y citar el texto correcto, no extraer cifras XBRL.
- Los filtros y, sobre todo, el query rewrite producen el salto de recuperación.
- El resultado debe evaluarse junto con trayectoria, coste y latencia.

Qué aprendimos al romperlo

- Un guardrail puede empeorar el sistema si impone XBRL a cifras que vienen legítimamente del texto.
- Las tablas fragmentadas pueden conservar el número y perder su unidad o contexto.
- El golden también forma parte del sistema: debe fijar qué significa `cifra` y cómo tratar eventos como splits.

Qué haríamos después

- Separar en la salida valor fuente, operación y cifra derivada.
- Comparar modelos de embeddings manteniendo el mismo golden y pipeline.
- Añadir preguntas de tablas, sumas entre filas y conceptos ausentes en XBRL.
- Evaluar retrieval y agente end-to-end por separado para saber dónde aparece cada mejora.

Una mejora cuenta cuando corrige un fallo observable, mueve una métrica y podemos poner su coste al lado del beneficio.